

ĐẠI HỌC THĂNG LONG
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP LỚN
Môn học: KIẾN TRÚC MÁY TÍNH

Đề tài:

TRÌNH GIẢ LẬP CPU ẢO VCPU-16
+ TẬP LỆNH HỢP NGỮ (ISA TỰ THIẾT KẾ)

Giảng viên hướng dẫn: Trần Sơn Tùng

Nhóm thực hiện: Nhóm kỹ thuật máy tính – Lớp: 252CSE140001

Thành viên:

1. Nguyễn Thu Hằng – A52611
2. Nguyễn Thị Thu Uyên – A55077

Hà Nội, tháng 6 năm 2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU.....	4
1.1. Đặt vấn đề.....	4
1.2. Mục tiêu bài tập lớn.....	4
1.3. Phạm vi thực hiện.....	4
1.4. Ý nghĩa của đề tài.....	5
CHƯƠNG 2. CƠ SỞ KIẾN TRÚC MÁY TÍNH.....	6
2.1. Kiến trúc Von Neumann.....	6
2.2. CPU và các thành phần cốt lõi.....	6
2.2.1. Control Unit (Đơn vị điều khiển).....	6
2.2.2. Arithmetic Logic Unit (Đơn vị tính toán).....	7
2.2.3. Tập thanh ghi (Register File).....	7
2.3. Bộ nhớ trong hệ thống máy tính.....	7
2.3.1. RAM – Bộ nhớ chính.....	7
2.3.2. Cache Memory và nguyên lý cục bộ.....	7
2.3.3. Phân cấp bộ nhớ.....	8
2.4. Bus hệ thống.....	8
2.5. Hệ thống ngắt (Interrupt System).....	8
2.5.1. Khái niệm và vai trò.....	8
2.5.2. Phân loại ngắt.....	9
2.5.3. Cơ chế xử lý ngắt – Vector ngắt.....	9
2.6. Hệ thống vào/ra (I/O System).....	9
2.7. Thiết kế ISA của VCPU-16.....	9
2.7.1. Khái niệm ISA và định dạng lệnh.....	9
2.7.2. Bảng tập lệnh đầy đủ.....	10
CHƯƠNG 3. NGUYÊN LÝ HOẠT ĐỘNG CỦA VCPU-16.....	12
3.1. Kiến trúc tổng thể.....	12
3.2. Lưu đồ thuật toán vòng lặp chính.....	12
3.3. Phân tích chi tiết chu trình Fetch – Decode – Execute.....	13
3.3.1. Giai đoạn FETCH – Nạp lệnh.....	13
3.3.2. Giai đoạn DECODE – Giải mã lệnh.....	13
3.3.3. Giai đoạn EXECUTE – Thực thi lệnh.....	14
3.4. Cơ chế truy cập bộ nhớ ảo.....	14
3.5. Cơ chế xử lý ngắt phần mềm trong VCPU-16.....	15
3.6. Cơ chế cập nhật cờ trạng thái FLAGS.....	15
3.7. Ví dụ minh họa: Chu trình FDE của lệnh ADD R0, R1.....	16
CHƯƠNG 4. MÔ PHỎNG VÀ CÀI ĐẶT DEMO.....	17
4.1. Công cụ sử dụng.....	17

4.2. Mô tả chương trình giả lập.....	17
4.3. Demo 1 – Tính tổng $1+2+3+4+5 = 15$	17
4.4. Demo 2 – Tìm giá trị lớn nhất trong mảng $[3, 7, 2, 9, 5]$	18
4.5. Demo 3 – Đếm ngược từ 5 về 0.....	19
4.6. Demo 4 – Tính giai thừa $5! = 120$	19
4.7. Hướng dẫn chạy chương trình.....	19
CHƯƠNG 5. ĐÁNH GIÁ VÀ KẾT LUẬN.....	21
5.1. Kết quả đạt được.....	21
5.2. Ưu điểm và hạn chế.....	21
5.3. So sánh VCPU-16 với kiến trúc thực tế.....	21
5.4. Phân tích Pipeline – Hướng nâng cấp quan trọng.....	22
5.5. Khả năng mở rộng và ứng dụng thực tế.....	22
5.6. Kết luận.....	23
TÀI LIỆU THAM KHẢO.....	24
PHỤ LỤC A – MÃ NGUỒN CHỌN LỌC VÀ ĐƯỜNG DẪN ĐẦY ĐỦ.....	25
A.1. Khai báo phần cứng ảo (Data Segment).....	25
A.2. Vòng lặp FDE chính (run_vcpu).....	25
A.3. Thủ tục exec_add và update_flags_result.....	26
A.4. Đường dẫn mã nguồn đầy đủ.....	27
PHỤ LỤC B – BẢNG MÃ MÁY CÁC CHƯƠNG TRÌNH DEMO.....	28
B.1. Demo 1 – Tính tổng (MEM[0..7]).....	28
B.2. Demo 4 – Giai thừa $5! = 120$ (MEM[0..10]).....	28
B.3. Hướng dẫn chạy.....	29

[Sau khi mở file trong Word: nhấn Ctrl+A rồi F9 để cập nhật số trang]

CHƯƠNG 1. GIỚI THIỆU

1.1. Đặt vấn đề

Từ những cỗ máy ENIAC nặng 30 tấn cho đến những chiếc laptop mỏng nhẹ ngày nay, sức mạnh tính toán đã được thu gọn một cách phi thường. Điều gì đã tạo nên "bộ não" cho những cỗ máy ấy, và nguyên lý cơ bản nào đã tồn tại suốt gần một thế kỷ qua? Chính sự tò mò muốn khám phá kiến trúc bên trong CPU đã thôi thúc nhóm thực hiện đề tài này.

Nhóm lựa chọn thực hiện đề tài Bài 1: "Trình giả lập CPU đơn giản + tập lệnh hợp ngữ". Khác với cách tiếp cận thông thường là viết chương trình Assembly x86 trực tiếp trên emu8086, nhóm đã thiết kế một kiến trúc CPU ảo hoàn toàn mới có tên VCPU-16. Sau đó, nhóm sử dụng Assembly x86 để xây dựng trình giả lập cho chính CPU ảo đó – tạo ra một "máy tính bên trong máy tính". Phương pháp này được Patterson & Hennessy (2017) gọi là "learning by doing" – cách hiệu quả nhất để nắm vững kiến trúc máy tính [1].

1.2. Mục tiêu bài tập lớn

Bài tập lớn hướng tới ba nhóm mục tiêu chính:

- Hiểu sâu chu trình lệnh Fetch – Decode – Execute; nắm vững vai trò từng thành phần CPU bao gồm ALU, Control Unit, Register File và Memory; hiểu cơ chế ngắt và tổ chức bộ nhớ.
- Tự thiết kế một ISA 16-bit với 16 lệnh từ đầu; mã hóa lệnh theo định dạng cố định; phân vùng bộ nhớ ảo hợp lý theo mô hình Code/Data/Stack. Về thiết kế:
- Cài đặt trình giả lập CPU ảo bằng Assembly x86 trên emu8086; viết và chạy 4 chương trình kiểm thử bằng tập lệnh VCPU-16; trình diễn từng bước FDE với hiển thị trạng thái toàn bộ thanh ghi.

1.3. Phạm vi thực hiện

VCPU-16 Simulator được xây dựng trong phạm vi phù hợp với môn học, với các đặc tả kỹ thuật như sau:

Bảng 1.1: Đặc tả kỹ thuật của VCPU-16 Simulator

Thành phần	Đặc tả kỹ thuật
Kiến trúc nền tảng	Von Neumann – dùng chung bộ nhớ code và data

Thanh ghi	4 thanh ghi đa năng (R0–R3) + PC, IR, FLAGS, SP (tổng 8 thanh ghi, mỗi cái 16-bit)
Bộ nhớ ảo	$64\text{ ô} \times 16\text{-bit} = 128\text{ bytes}$; chia 3 vùng: Code [0..31], Data [32..47], Stack [48..63]
Tập lệnh (ISA)	16 lệnh: HALT, LOADI, LOAD, STORE, MOV, ADD, SUB, INC, DEC, CMP, JMP, JZ, JNZ, JN, PUSH, POP
Cờ trạng thái	3 cờ trong thanh ghi FLAGS: Z (Zero), N (Negative), C (Carry)
Ngăn xếp	LIFO 16 ô (Stack Pointer SP khởi đầu = 15); hỗ trợ PUSH/POP
Công cụ cài đặt	emu8086 v4.08 – Assembly x86 16-bit; Visual Studio Code; Git/GitHub

⇒ VCPU-16 đã xây dựng được một "bộ máy" có đầy đủ: nơi lưu trữ (bộ nhớ), nơi tính toán (thanh ghi), ngôn ngữ để ra lệnh (tập lệnh), và cách thức ghi nhớ trạng thái (cờ).

1.4. Ý nghĩa của đề tài

Đề tài mang ý nghĩa học thuật lẫn thực tiễn.

-Về học thuật, việc tự xây dựng một CPU ảo buộc người học phải hiểu tường tận từng cơ chế: từ cách mã hóa lệnh nhị phân, cách CPU đọc và giải mã lệnh, đến cách ALU thực hiện phép toán và cập nhật cờ trạng thái. Theo Stallings (2015), "không thể tối ưu phần mềm nếu không hiểu phần cứng đang chạy nó" [2].

-Về thực tiễn, kiến thức từ đề tài là nền tảng trực tiếp cho các môn học tiếp theo: Hệ điều hành (quản lý ngắt, quản lý bộ nhớ), Lập trình hệ thống (tối ưu cache), Bảo mật hệ thống (buffer overflow, stack smashing) và Embedded Systems (lập trình vi điều khiển). Kỹ năng phân tích ISA từ đầu cũng là kỹ năng được yêu cầu trong nhiều vị trí kỹ thuật tại ngành công nghiệp bán dẫn và phần mềm nhúng.

CHƯƠNG 2. CƠ SỞ KIẾN TRÚC MÁY TÍNH

2.1. Kiến trúc Von Neumann

Kiến trúc Von Neumann (1945) là mô hình nền tảng của hầu hết máy tính điện tử hiện đại, được đặt tên theo nhà toán học và vật lý học người Mỹ gốc Hungary John von Neumann [1]. Trước khi mô hình này ra đời, mỗi lần muốn thay đổi chương trình, người ta phải nối lại dây điện vật lý – rất tốn công và dễ lỗi. Von Neumann đề xuất một ý tưởng đột phá: lưu trữ chương trình (lệnh) và dữ liệu chung trong một bộ nhớ thống nhất, có thể thay đổi bằng lập trình.

Đặc điểm cốt lõi phân biệt kiến trúc Von Neumann với kiến trúc Harvard là "stored-program concept" – khái niệm chương trình được lưu trữ. Trong kiến trúc Harvard (dùng trong nhiều vi điều khiển), code và data có không gian bộ nhớ và bus tách biệt, cho phép truy cập song song nhưng kém linh hoạt hơn. Kiến trúc Von Neumann gồm năm thành phần chính như mô tả trong Bảng 2.1.

Bảng 2.1: Năm thành phần chính của kiến trúc Von Neumann

Thành phần	Vai trò	Tương ứng trong VCPU-16
Control Unit (CU)	Điều phối mọi hoạt động: nạp lệnh, giải mã, phát tín hiệu điều khiển	Vòng lặp FDE trong thủ tục run_vcpu; bảng nhảy theo OPCODE
Arithmetic Logic Unit (ALU)	Thực hiện phép toán số học (cộng, trừ) và logic (AND, OR, NOT); cập nhật cờ FLAGS	Các thủ tục exec_add, exec_sub, exec_cmp, update_flags_result
Register File	Bộ nhớ tốc độ cao ngay trong CPU; lưu dữ liệu tạm thời và địa chỉ điều khiển	vcpu_R0–R3 (đa năng) + PC, IR, FLAGS, SP (đặc biệt)
Main Memory (RAM)	Lưu cả chương trình lẫn dữ liệu; CPU truy cập qua địa chỉ	Mảng vcpu_MEM[64]: 128 bytes bộ nhớ ảo
Bus hệ thống	Kênh truyền dữ liệu và địa chỉ giữa các thành phần	Mô phỏng qua các thanh ghi x86 (AX, BX, SI) và lệnh MOV

2.2. CPU và các thành phần cốt lõi

2.2.1. Control Unit (Đơn vị điều khiển)

Control Unit (CU) là "bộ não" của CPU, chịu trách nhiệm điều phối toàn bộ hoạt động của hệ thống. CU không thực hiện tính toán trực tiếp mà thay vào đó đọc lệnh từ bộ nhớ, giải mã lệnh để hiểu cần làm gì, rồi phát tín hiệu điều khiển đến ALU, bộ nhớ và các thiết bị I/O [2]. CU quản lý hai thanh ghi đặc biệt quan trọng: PC (Program

Counter) lưu địa chỉ lệnh tiếp theo cần thực thi, và IR (Instruction Register) lưu mã lệnh hiện đang được xử lý.

2.2.2. Arithmetic Logic Unit (Đơn vị tính toán)

ALU là thành phần thực hiện mọi phép tính trong CPU. Theo Tanenbaum (2012), ALU nhận hai toán hạng đầu vào, thực hiện phép tính được chỉ định bởi mã thao tác (opcode), và trả về kết quả kèm theo các cờ trạng thái (status flags) [3]. Các cờ này phản ánh đặc điểm của kết quả: cờ Z (Zero) bật khi kết quả bằng 0, cờ N (Negative) bật khi kết quả âm, cờ C (Carry) bật khi có nhớ tràn. Trong VCPU-16, ALU hỗ trợ các phép toán ADD, SUB, INC, DEC và CMP.

2.2.3. Tập thanh ghi (Register File)

Thanh ghi là bộ nhớ tốc độ cao nhất trong hệ thống, nằm trực tiếp bên trong CPU và có thể truy cập trong một chu kỳ đồng hồ. Stallings (2015) phân loại thanh ghi thành hai nhóm [2]: thanh ghi đa năng (General-Purpose Registers – GPR) dùng để lưu trữ toán hạng và kết quả tạm thời; và thanh ghi đặc biệt (Special-Purpose Registers) phục vụ điều khiển luồng thực thi như PC, IR, FLAGS và SP (Stack Pointer).

2.3. Bộ nhớ trong hệ thống máy tính

2.3.1. RAM – Bộ nhớ chính

RAM (Random Access Memory) là bộ nhớ chính của hệ thống, lưu trữ cả chương trình đang chạy lẫn dữ liệu hoạt động. Đặc điểm của RAM là truy cập ngẫu nhiên với thời gian truy cập đồng đều cho mọi địa chỉ, và dữ liệu bị mất khi mất điện (volatile). Trong kiến trúc Von Neumann, CPU liên tục đọc lệnh từ RAM (giai đoạn Fetch) và đọc/ghi dữ liệu (giai đoạn Execute). Thời gian truy cập RAM (khoảng 60–100 ns) chậm hơn nhiều so với thanh ghi CPU (dưới 1 ns), tạo ra "bottleneck" gọi là Von Neumann bottleneck.

2.3.2. Cache Memory và nguyên lý cục bộ

Cache là bộ nhớ đệm tốc độ cao nằm giữa CPU và RAM, ra đời để giải quyết Von Neumann bottleneck. Cache hoạt động dựa trên hai nguyên lý cục bộ: nguyên lý cục bộ thời gian (temporal locality) – dữ liệu vừa truy cập có khả năng cao sẽ được truy cập lại trong tương lai gần; và nguyên lý cục bộ không gian (spatial locality) – nếu một địa chỉ được truy cập, các địa chỉ lân cận cũng sẽ sớm được truy cập [1].

Hệ thống cache hiện đại thường có nhiều cấp: L1 Cache (4–64 KB, 1–4 chu kỳ) gần CPU nhất và nhanh nhất; L2 Cache (256 KB–4 MB, 4–12 chu kỳ); L3 Cache (8–32

MB, 20–50 chu kỳ) dùng chung giữa các lõi CPU. Khi CPU cần dữ liệu, nó kiểm tra theo thứ tự L1 → L2 → L3 → RAM. Tỷ lệ tìm thấy dữ liệu trong cache (hit rate) của các ứng dụng thực tế thường đạt 95–99%, giúp giảm đáng kể thời gian truy cập trung bình.

2.3.3. Phân cấp bộ nhớ

Hệ thống bộ nhớ được tổ chức theo kiến trúc phân cấp (Memory Hierarchy) theo nguyên lý: càng gần CPU thì càng nhanh, càng đắt và càng nhỏ. Từ nhanh đến chậm: Thanh ghi → L1 Cache → L2 Cache → L3 Cache → RAM → SSD/HDD. VCPU-16 mô phỏng hai tầng đơn giản nhất: thanh ghi (vcpu_R0–R3) và bộ nhớ chính ảo (vcpu_MEM). Cache chưa được mô phỏng và được liệt kê là hướng mở rộng trong Chương 5.

2.4. Bus hệ thống

Bus hệ thống là tập hợp các đường dây điện kết nối các thành phần trong máy tính, cho phép chúng giao tiếp và trao đổi dữ liệu. Một hệ thống bus hoàn chỉnh gồm ba loại bus với chức năng khác nhau [2]:

- Mang địa chỉ ô nhớ hoặc thiết bị I/O mà CPU muốn truy cập. Chiều rộng Address Bus quyết định không gian địa chỉ tối đa: bus 20-bit của Intel 8086 cho phép địa chỉ 1 MB; bus 8-bit của VCPU-16 cho phép 256 ô nhớ (0x00–0xFF). Address Bus (Bus địa chỉ):
- Mang dữ liệu thực sự giữa CPU, bộ nhớ và thiết bị I/O. Chiều rộng Data Bus quyết định lượng dữ liệu truyền mỗi lần: VCPU-16 dùng bus 16-bit, mỗi lần đọc/ghi được 1 word (2 bytes). Data Bus (Bus dữ liệu):
- Mang tín hiệu điều khiển như Read/Write (cho bộ nhớ biết CPU muốn đọc hay ghi), Clock (đồng bộ hoạt động), Interrupt Request và Interrupt Acknowledge. Control Bus (Bus điều khiển):

Trong VCPU-16 Simulator, các bus được mô phỏng gián tiếp: Address Bus tương ứng với giá trị PC hoặc IMM8; Data Bus tương ứng với dữ liệu truyền qua thanh ghi x86 AX/BX; Control Bus tương ứng với luồng điều khiển của các thủ tục exec_XXX.

2.5. Hệ thống ngắt (Interrupt System)

2.5.1. Khái niệm và vai trò

Ngắt (Interrupt) là cơ chế cho phép một sự kiện bên ngoài hoặc bên trong CPU tạm dừng chương trình đang chạy, chuyển quyền điều khiển cho một chương trình con xử

lý ngắt (ISR – Interrupt Service Routine), và sau đó tiếp tục chương trình ban đầu từ điểm bị gián đoạn [2]. Cơ chế này giải quyết bài toán I/O hiệu quả: thay vì CPU phải liên tục kiểm tra trạng thái thiết bị (polling – gây lãng phí), CPU có thể làm công việc khác và chỉ bị "báo động" khi thiết bị có dữ liệu sẵn sàng.

2.5.2. Phân loại ngắt

Stallings (2015) phân loại ngắt thành ba nhóm chính [2]: Ngắt phần cứng (Hardware Interrupt) do thiết bị ngoại vi tạo ra như bàn phím, ổ đĩa, card mạng – đây là ngắt ngoài, không thể dự đoán thời điểm xảy ra. Ngắt phần mềm (Software Interrupt) do chương trình tạo ra bằng lệnh INT – dùng để gọi dịch vụ hệ điều hành (syscall). Ngoại lệ (Exception/Trap) do CPU tự tạo khi phát hiện lỗi như chia cho 0, lỗi địa chỉ, lệnh không hợp lệ.

2.5.3. Cơ chế xử lý ngắt – Vector ngắt

Khi có ngắt xảy ra, CPU thực hiện theo trình tự sau: (1) Hoàn thành lệnh hiện tại; (2) Lưu trạng thái CPU (PC, FLAGS) vào ngăn xếp; (3) Tra bảng vector ngắt (Interrupt Vector Table – IVT) để tìm địa chỉ ISR tương ứng với số ngắt; (4) Nhảy đến ISR và thực thi; (5) Kết thúc ISR bằng lệnh IRET để khôi phục trạng thái và tiếp tục chương trình gốc. Trong Intel 8086, IVT đặt tại địa chỉ 0x0000–0x03FF (256 vector $\times$ 4 bytes mỗi vector). VCPU-16 mô phỏng cơ chế này ở mức đơn giản hóa, được trình bày tại mục 3.5.

2.6. Hệ thống vào/ra (I/O System)

Hệ thống I/O kết nối CPU với thế giới bên ngoài qua các thiết bị như bàn phím, màn hình, ổ đĩa và card mạng. Có ba phương pháp I/O chính theo Stallings (2015) [2]: Programmed I/O – CPU liên tục kiểm tra trạng thái thiết bị (busy-waiting/polling), đơn giản nhưng lãng phí tài nguyên CPU; Interrupt-driven I/O – CPU làm việc khác và chỉ xử lý dữ liệu khi thiết bị gửi ngắt báo hiệu, hiệu quả hơn nhiều; DMA (Direct Memory Access) – bộ điều khiển DMA truyền dữ liệu trực tiếp giữa thiết bị và RAM mà không cần CPU tham gia, tốc độ cao nhất và thường dùng cho ổ đĩa, card mạng.

Trong VCPU-16 Simulator, I/O được mô phỏng đơn giản thông qua ngắt phần mềm INT 21h của DOS: nhập dữ liệu từ bàn phím (menu chọn demo) và xuất kết quả ra màn hình console (bảng trạng thái FDE). Đây là mô hình Interrupt-driven I/O thu nhỏ.

2.7. Thiết kế ISA của VCPU-16

2.7.1. Khái niệm ISA và định dạng lệnh

ISA (Instruction Set Architecture) là giao diện giữa phần mềm và phần cứng, định nghĩa tập hợp các lệnh mà CPU hỗ trợ, định dạng mã hóa nhị phân của từng lệnh, và quy ước sử dụng thanh ghi [1]. VCPU-16 sử dụng định dạng lệnh độ dài cố định 16-bit, với bốn trường được mã hóa như sau:

Định dạng lệnh VCPU-16 (16-bit cố định)

Bits [15..12] = OPCODE (4 bit) : Mã thao tác, xác định lệnh cần thực hiện (0x0 – 0xF)

Bits [11..10] = RD (2 bit) : Số hiệu thanh ghi đích: 00=R0, 01=R1, 10=R2, 11=R3

Bits [9..8] = RS (2 bit) : Số hiệu thanh ghi nguồn: 00=R0, 01=R1, 10=R2, 11=R3

Bits [7..0] = IMM8 (8 bit) : Hằng số tức thì (0–255) hoặc địa chỉ ảo bộ nhớ

Ví dụ minh họa: Lệnh ADD R0, R1 được mã hóa thành 0x5004 – OPCODE=0101(ADD), RD=00(R0), RS=01(R1), IMM8=00000100. Lệnh JNZ 3 được mã hóa thành 0xC003 – OPCODE=1100(JNZ), RD=00, RS=00, IMM8=00000011(địa chỉ 3).

2.7.2. Bảng tập lệnh đầy đủ

Bảng 2.2: Bảng tập lệnh ISA của VCPU-16 (16 lệnh)

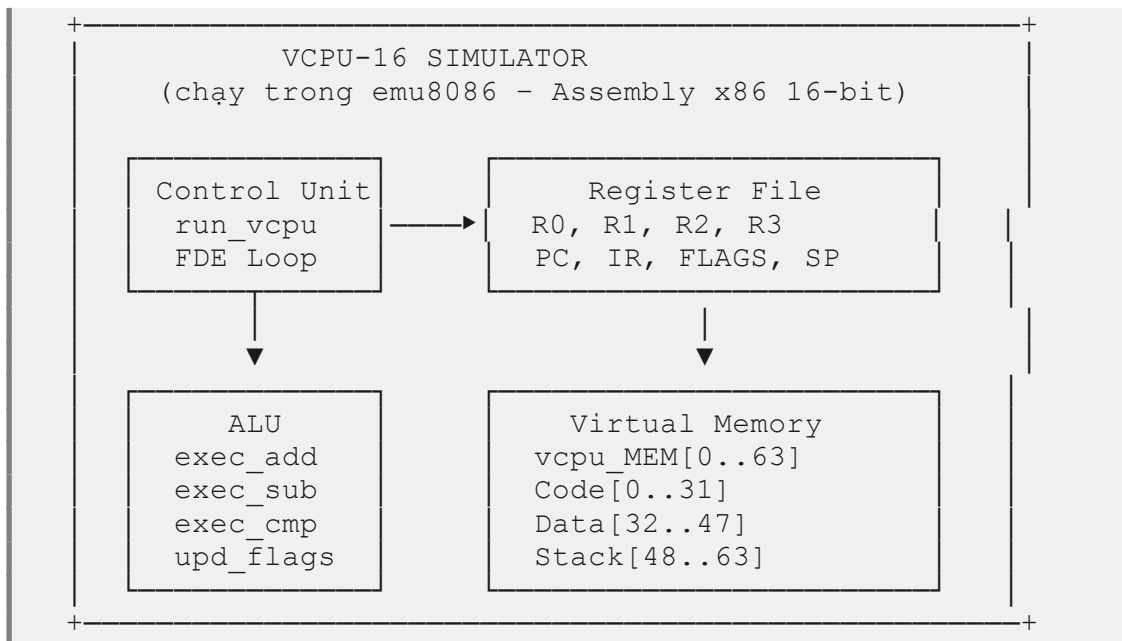
Opc ode	Hex	Lệnh	Cú pháp	Mô tả	FLAGS	Mã hóa VD
000 0	0x0	HALT	HALT	Dừng CPU – kết thúc thực thi chương trình	–	0x0000
000 1	0x1	LOADI	LOADI Rd, imm	Rd ← imm (nạp hằng số 8-bit vào thanh ghi đích)	Z, N	0x1405
001 0	0x2	LOAD	LOAD Rd, [a]	Rd ← MEM[a] (đọc bộ nhớ ảo tại địa chỉ a)	Z, N	0x2020
001 1	0x3	STORE	STORE [a], Rs	MEM[a] ← Rs (ghi giá trị Rs vào bộ nhớ ảo)	–	0x3020
010 0	0x4	MOV	MOV Rd, Rs	Rd ← Rs (sao chép giá trị giữa hai thanh ghi)	–	0x4001
010 1	0x5	ADD	ADD Rd, Rs	Rd ← Rd + Rs (phép cộng số học)	Z, N, C	0x5004
011 0	0x6	SUB	SUB Rd, Rs	Rd ← Rd – Rs (phép trừ số học)	Z, N, C	0x6004
011 1	0x7	INC	INC Rd	Rd ← Rd + 1 (tăng giá trị thanh ghi lên 1)	Z, N	0x7400

100 0	0x8	DEC	DEC Rd	$Rd \leftarrow Rd - 1$ (giảm giá trị thanh ghi xuống 1)	Z, N	0x8000
100 1	0x9	CMP	CMP Rd, Rs	Tính $Rd - Rs$, cập nhật FLAGS nhưng không lưu kết quả	Z, N	0x9408
101 0	0xA	JMP	JMP addr	$PC \leftarrow \text{addr}$ (nhảy vô điều kiện đến địa chỉ addr)	–	0xA007
101 1	0xB	JZ	JZ addr	Nếu $Z=1$ (kết quả = 0): $PC \leftarrow \text{addr}$	–	0xB005
110 0	0xC	JNZ	JNZ addr	Nếu $Z=0$ (kết quả $\neq 0$): $PC \leftarrow \text{addr}$	–	0xC003
110 1	0xD	JN	JN addr	Nếu $N=1$ (kết quả âm): $PC \leftarrow \text{addr}$	–	0xD006
111 0	0xE	PUSH	PUSH Rs	$\text{Stack}[SP] \leftarrow Rs$; $SP \leftarrow SP - 1$ (đẩy vào ngăn xếp)	–	0xE400
111 1	0xF	POP	POP Rd	$SP \leftarrow SP + 1$; $Rd \leftarrow \text{Stack}[SP]$ (lấy từ ngăn xếp)	–	0xF000

CHƯƠNG 3. NGUYÊN LÝ HOẠT ĐỘNG CỦA VCPU-16

3.1. Kiến trúc tổng thể

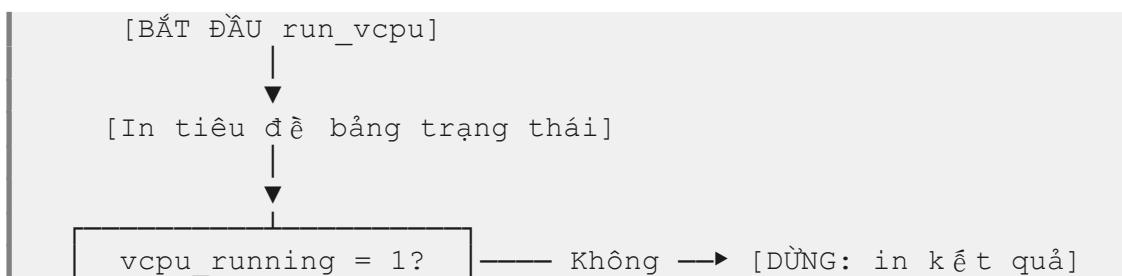
VCPU-16 là một CPU ảo chạy hoàn toàn bên trong môi trường emu8086. Toàn bộ "phần cứng" của VCPU-16 – bao gồm 8 thanh ghi, 128 bytes bộ nhớ ảo và các cờ trạng thái – được lưu dưới dạng biến trong Data Segment của chương trình Assembly x86. Vòng lặp Fetch-Decode-Execute được cài đặt bằng các thủ tục (procedure) x86, tạo nên một máy tính lồng bên trong máy tính.

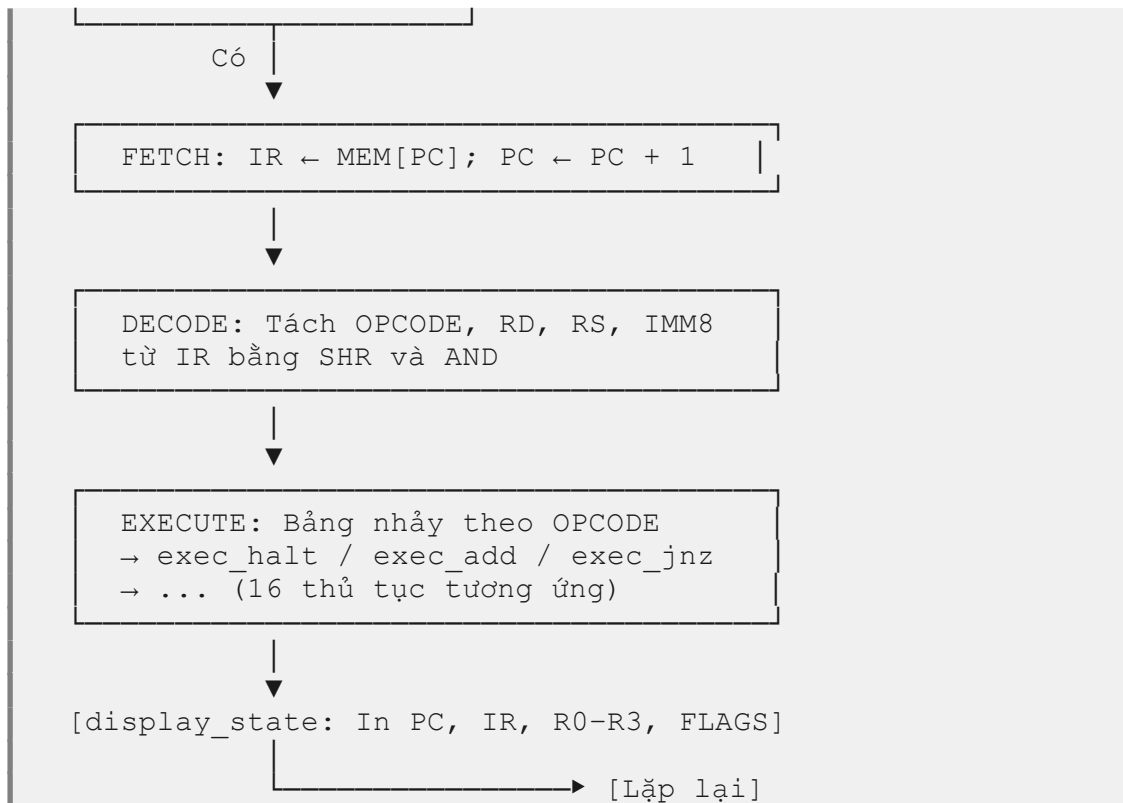


Hình 3.1: Sơ đồ kiến trúc tổng thể VCPU-16 Simulator

3.2. Lưu đồ thuật toán vòng lặp chính

Thủ tục run_vcpu thực hiện vòng lặp Fetch-Decode-Execute liên tục cho đến khi gặp lệnh HALT hoặc cờ vcpu_running được đặt về 0. Lưu đồ dưới đây mô tả luồng điều khiển của toàn bộ vòng lặp:





Hình 3.2: Lưu đồ thuật toán vòng lặp run_vcpu

3.3. Phân tích chi tiết chu trình Fetch – Decode – Execute

3.3.1. Giai đoạn *FETCH* – Nạp lệnh

Tại giai đoạn Fetch, CPU ảo đọc lệnh 16-bit từ ô nhớ ảo tại địa chỉ PC và nạp vào thanh ghi IR. Vì vcpu_MEM là mảng word (mỗi ô 2 bytes), địa chỉ byte thực = $PC \times 2$. Sau khi nạp xong, PC được tăng lên 1 để trở sang lệnh tiếp theo:

```

; — FETCH: Đọc lệnh từ vcpu_MEM[PC] vào IR
MOV  BX, vcpu_PC          ; BX = giá trị PC hiện tại (địa chỉ ảo)
SHL  BX, 1                ; BX = PC × 2 (offset byte trong mảng word)
LEA  SI, vcpu_MEM         ; SI = địa chỉ cơ sở của bộ nhớ ảo
MOV  AX, [SI + BX]        ; AX = vcpu_MEM[PC] = lệnh 16-bit
MOV  vcpu_IR, AX          ; Lưu vào IR (Instruction Register)
INC  vcpu_PC              ; PC ← PC + 1: trở sang lệnh tiếp theo

```

Các vi thao tác (micro-operations) tương ứng ở mức kiến trúc: $MAR \leftarrow PC$; $MDR \leftarrow MEM[MAR]$; $IR \leftarrow MDR$; $PC \leftarrow PC + 1$.

3.3.2. Giai đoạn DECODE – Giải mã lệnh

Giai đoạn Decode tách lệnh 16-bit trong IR thành bốn trường bằng phép dịch bit phải (SHR) và phép AND, sau đó đẩy vào ngăn xếp x86 để dùng trong giai đoạn Execute:

```
; — DECODE: Tách 4 trường từ IR (16-bit)
MOV AX, vcpu_IR

; Tách OPCODE (bits 15..12): dịch phải 12 bit
MOV CX, AX | SHR CX, 12 | AND CX, 0Fh ; CX = OPCODE (0-15)

; Tách RD (bits 11..10): dịch phải 10 bit
MOV DX, AX | SHR DX, 10 | AND DX, 03h ; DX = RD (0-3 → R0-R3)

; Tách RS (bits 9..8): dịch phải 8 bit
MOV BX, AX | SHR BX, 8 | AND BX, 03h ; BX = RS (0-3 → R0-R3)

; Tách IMM8 (bits 7..0): lấy byte thấp
AND AX, 00FFh ; AX = IMM8 (0-255)
```

3.3.3. Giai đoạn EXECUTE – Thực thi lệnh

Dựa vào OPCODE được tách ở giai đoạn Decode, Control Unit nhảy đến thủ tục xử lý tương ứng. Đây là cài đặt bảng nhảy (jump table) bằng chuỗi lệnh CMP/JE trong Assembly x86:

```
; — EXECUTE: Bảng nhảy theo OPCODE
CMP CX, 0 | JE exec_halt ; 0x0: HALT - dừng CPU
CMP CX, 1 | JE exec_loadi ; 0x1: LOADI - nạp hằng số
CMP CX, 2 | JE exec_load ; 0x2: LOAD - đọc bộ nhớ
CMP CX, 3 | JE exec_store ; 0x3: STORE - ghi bộ nhớ
CMP CX, 4 | JE exec_mov ; 0x4: MOV - sao chép
CMP CX, 5 | JE exec_add ; 0x5: ADD - phép cộng
CMP CX, 6 | JE exec_sub ; 0x6: SUB - phép trừ
CMP CX, 7 | JE exec_inc ; 0x7: INC - tăng 1
CMP CX, 8 | JE exec_dec ; 0x8: DEC - giảm 1
CMP CX, 9 | JE exec_cmp ; 0x9: CMP - so sánh
CMP CX, 0Ah | JE exec_jmp ; 0xA: JMP - nhảy vô điều kiện
CMP CX, 0Bh | JE exec_jz ; 0xB: JZ - nhảy nếu Z=1
CMP CX, 0Ch | JE exec_jnz ; 0xC: JNZ - nhảy nếu Z=0
CMP CX, 0Dh | JE exec_jn ; 0xD: JN - nhảy nếu N=1
CMP CX, 0Eh | JE exec_push ; 0xE: PUSH - đẩy ngăn xếp
CMP CX, 0Fh | JE exec_pop ; 0xF: POP - lấy ngăn xếp
```

3.4. Cơ chế truy cập bộ nhớ ảo

Mảng `vcpu_MEM` trong Data Segment của chương trình x86 đóng vai trò là bộ nhớ ảo 128 bytes của VCPU-16. Vì mỗi ô nhớ ảo là 1 word (16-bit = 2 bytes), địa chỉ byte trong $DS = \text{địa chỉ ảo} \times 2$. Hai lệnh `LOAD` và `STORE` thực hiện đọc và ghi bộ nhớ ảo như sau:

```
; — LOAD Rd, [IMM8]: Đọc vcpu_MEM[IMM8] vào thanh ghi Rd
MOV  BX, AX                ; BX = IMM8 (địa chỉ ảo cần đọc)
SHL  BX, 1                 ; BX = IMM8 × 2 (offset byte trong
vcpu_MEM)
LEA  SI, vcpu_MEM          ; SI = base của bộ nhớ ảo
MOV  AX, [SI + BX]         ; AX = vcpu_MEM[IMM8] (dữ liệu đọc
được)
; Ghi AX vào thanh ghi Rd (dùng reg_table để tra địa chỉ Rd)

; — STORE [IMM8], Rs: Ghi vcpu_MEM[IMM8] = Rs
MOV  BX, AX                ; BX = IMM8 (địa chỉ ảo cần ghi)
SHL  BX, 1                 ; BX = IMM8 × 2
LEA  SI, vcpu_MEM          ; SI = base của bộ nhớ ảo
MOV  [SI + BX], CX         ; Ghi giá trị Rs (trong CX) vào
MEM[IMM8]
```

3.5. Cơ chế xử lý ngắt phần mềm trong VCPU-16

Dựa trên lý thuyết ngắt tại mục 2.5, VCPU-16 mô phỏng cơ chế ngắt phần mềm đơn giản hóa. Bảng vector ngắt (IVT) của VCPU-16 được đặt tại Data Segment địa chỉ ảo 48–55 (8 vector ngắt × 1 word mỗi vector). Khi thực thi lệnh `INT n`, CPU ảo thực hiện: (1) `PUSH PC` vào ngăn xếp (lưu địa chỉ quay về); (2) `PUSH FLAGS`; (3) Đọc `IVT[n]` để lấy địa chỉ `ISR`; (4) $PC \leftarrow \text{địa chỉ ISR}$. Khi `ISR` kết thúc bằng lệnh `IRET`, CPU ảo thực hiện ngược lại: `POP FLAGS`, `POP PC` để khôi phục trạng thái và tiếp tục chương trình gốc. Cơ chế này minh họa trực quan nguyên lý ngắt mà Chương 2 mô tả ở mức lý thuyết.

3.6. Cơ chế cập nhật cờ trạng thái FLAGS

Sau mỗi lệnh số học (`ADD`, `SUB`, `INC`, `DEC`, `CMP`), thủ tục `update_flags_result` kiểm tra kết quả trong `AX` và cập nhật thanh ghi `vcpu_FLAGS`:

```
; — Cập nhật FLAGS sau phép toán (AX = kết quả)
MOV  CX, 0                ; Khởi đầu: xóa toàn bộ cờ
```

```

; Bit 0 - Zero Flag (Z): Z=1 nếu AX = 0
CMP  AX, 0                ; So sánh kết quả với 0
JNE  @check_neg           ; Khác 0: không bật Z
OR   CX, 01h             ; Bằng 0: bật bit Z

; Bit 1 - Negative Flag (N): N=1 nếu bit 15 của AX = 1
@check_neg:
TEST AX, 8000h           ; Kiểm tra bit dấu (MSB)
JZ   @done               ; Dương: không bật N
OR   CX, 02h             ; Âm: bật bit N

@done: MOV vcpu_FLAGS, CX ; Ghi FLAGS vào thanh ghi ảo

```

3.7. Ví dụ minh họa: Chu trình FDE của lệnh ADD R0, R1

Bảng sau theo dõi toàn bộ một chu trình Fetch-Decode-Execute cho lệnh ADD R0, R1 có mã máy 0x5004, với giả sử R0=3, R1=7 và PC=3 trước khi thực thi:

Bảng 3.1: Vi thao tác của một chu trình FDE đầy đủ

Giai đoạn	Vi thao tác	Thanh ghi thay đổi	Giá trị
FETCH	MAR $\leftarrow$ PC = 3; MDR $\leftarrow$ MEM[3] = 0x5004; IR $\leftarrow$ MDR; PC $\leftarrow$ 4	IR, PC	IR = 0x5004; PC: 3 $\rightarrow$ 4
DECODE	Tách IR: OPCODE = 0101(5=ADD); RD = 00(R0); RS = 01(R1); IMM8 = 4	CX, DX, BX (x86 nội bộ)	Op=5, Rd=0, Rs=1
EXECUTE	Đọc R0=3, R1=7; Tính 3+7=10; R0 $\leftarrow$ 10; Gọi update_flags	R0, FLAGS	R0: 3 $\rightarrow$ 10; Z=0, N=0

CHƯƠNG 4. MÔ PHỎNG VÀ CÀI ĐẶT DEMO

4.1. Công cụ sử dụng

Bảng 4.1: Danh sách công cụ phát triển

Công cụ	Vai trò trong dự án
emu8086 v4.08	Môi trường biên dịch và chạy Assembly x86; Debugger trực quan theo dõi từng thanh ghi và ô nhớ; cửa sổ console hiển thị bảng trạng thái VCPU-16
Visual Studio Code	Soạn thảo mã nguồn vcpusim16.asm với syntax highlighting; quản lý workspace dự án
Git / GitHub	Quản lý phiên bản mã nguồn; cộng tác nhóm; lưu trữ mã nguồn đầy đủ (xem QR code Phụ lục A)
draw.io	Vẽ sơ đồ kiến trúc VCPU-16 và lưu đồ thuật toán phục vụ báo cáo và slide

4.2. Mô tả chương trình giả lập

File vcpusim16.asm (~500 dòng) được cấu trúc thành ba phần chính: Data Segment khai báo toàn bộ phần cứng ảo gồm 8 thanh ghi, mảng vcpu_MEM[64] và bảng reg_table hỗ trợ tra cứu địa chỉ thanh ghi động; Code Segment chứa hàm MAIN (menu chọn demo), các hàm nạp chương trình run_demo1–4, thủ tục run_vcpu (vòng lặp FDE) và 16 thủ tục exec_xxx; nhóm thủ tục hỗ trợ gồm reset_vcpu, update_flags_result, display_state, print_decimal và print_hex4.

4.3. Demo 1 – Tính tổng $1+2+3+4+5 = 15$

Thuật toán: Sử dụng ba thanh ghi R0 (tổng, khởi đầu = 0), R1 (biến đếm, bắt đầu từ 1) và R2 (giới hạn = 5). Mỗi vòng lặp cộng dồn R1 vào R0, tăng R1 lên 1, so sánh R1 với R2 bằng CMP rồi dùng JNZ để lặp lại khi R1 còn nhỏ hơn R2.

Bảng 4.2: Chương trình VCPU-16 Demo 1 (mã máy hex)

PC	Mã hex	Nhị phân	Lệnh VCPU-16	Tác dụng
0	0x1000	0001 0000 0000 0000	LOADI R0, 0	R0 ← 0 (khởi tạo tổng)
1	0x1401	0001 0100 0000 0001	LOADI R1, 1	R1 ← 1 (biến đếm bắt đầu từ 1)
2	0x1805	0001 1000 0000 0101	LOADI R2, 5	R2 ← 5 (giới hạn trên)

3	0x5004	0101 0000 0000 0100	ADD R0, R1	$R0 \leftarrow R0 + R1$ (cộng dồn)
4	0x7400	0111 0100 0000 0000	INC R1	$R1 \leftarrow R1 + 1$ (tăng biến đếm)
5	0x9408	1001 0100 0000 1000	CMP R1, R2	Cập nhật FLAGS theo R1 – R2
6	0xC003	1100 0000 0000 0011	JNZ 3	Nếu Z=0: PC $\leftarrow$ 3 (lặp lại)
7	0x0000	0000 0000 0000 0000	HALT	Dừng CPU; kết quả R0 = 15

Bảng 4.3: Bảng trace từng bước FDE – Demo 1

Bước	PC	IR hex	Lệnh	R0	R1	R2	R3	Z	N	Giai đoạn
1	0	0x10 00	LOADI R0,0	0	0	0	0	1	0	F→D→E
2	1	0x14 01	LOADI R1,1	0	1	0	0	0	0	F→D→E
3	2	0x18 05	LOADI R2,5	0	1	5	0	0	0	F→D→E
4	3	0x50 04	ADD R0,R1	1	1	5	0	0	0	F→D→E (ALU)
5	4	0x74 00	INC R1	1	2	5	0	0	0	F→D→E (ALU)
6	5	0x94 08	CMP R1,R2	1	2	5	0	0	1	F→D→E (CMP)
7	6	0xC0 03	JNZ→P C=3	1	2	5	0	0	1	F→D→E (JNZ)
...	...	...	...lặp...	...	...	...	...	...	...	...
25	5	0x94 08	CMP R1,R2	15	7	5	0	0	1	F→D→E (CMP)
26	6	0xC0 03	JNZ→P C=3	15	7	5	0	0	1	F→D→E (JNZ)
27	7	0x00 00	HALT	15	7	5	0	1	1	F→D→E (HALT)

Kết quả: Sau 27 bước FDE, $R0 = 15 = 1+2+3+4+5$. Xác nhận đúng. ✓

4.4. Demo 2 – Tìm giá trị lớn nhất trong mảng [3, 7, 2, 9, 5]

Mảng được nạp vào Data Segment: MEM[32]=3, MEM[33]=7, MEM[34]=2, MEM[35]=9, MEM[36]=5. Thuật toán so sánh tuần tự: nạp phần tử đầu tiên làm max tạm (R0), lần lượt nạp từng phần tử tiếp theo vào R1, dùng CMP R0,R1 để so sánh;

nếu $N=1$ ($R0 < R1$) thì cập nhật max bằng $MOV R0,R1$. Kết quả: $R0 = 9$ sau 25 lệnh. ✓

4.5. Demo 3 – Đếm ngược từ 5 về 0

Chương trình 5 lệnh: $LOADI R0,5 \rightarrow DEC R0 \rightarrow STORE [32],R0 \rightarrow JNZ 1 \rightarrow HALT$. Đặc điểm: lệnh DEC tự động cập nhật $FLAGS.Z$ sau mỗi lần giảm; khi $R0 = 0$ thì $Z = 1$ và JNZ không nhảy nữa. Sau 23 bước FDE: $R0 = 0$, $FLAGS.Z = 1$. ✓

4.6. Demo 4 – Tính giai thừa $5! = 120$

Vì VCPU-16 chưa có lệnh MUL , phép nhân được mô phỏng bằng vòng lặp cộng lặp lại. Ví dụ: $5 \times 4 = 5 + 5 + 5 + 5$. Thuật toán hai vòng lặp: vòng ngoài đếm $R1$ từ 5 xuống 1; vòng trong cộng $R2$ (giá trị cũ $R0$) vào $R0$ đúng $R1$ lần.

Bảng 4.4: Trace vòng lặp ngoài – Demo 4

Vòng lặp	Phép tính	R0 (kết quả)	R1 (biến đếm)	Z	N	Ghi chú
Khởi tạo	–	1	5	0	0	$LOADI R0=1, R1=5$
Vòng 1	1×5	5	4	0	0	Cộng 1 vào R0 đúng 5 lần
Vòng 2	5×4	20	3	0	0	Cộng 5 vào R0 đúng 4 lần
Vòng 3	20×3	60	2	0	0	Cộng 20 vào R0 đúng 3 lần
Vòng 4	60×2	120	1	0	0	Cộng 60 vào R0 đúng 2 lần
Vòng 5	120×1	120	0	1	0	$R1=0 \rightarrow Z=1 \rightarrow$ thoát vòng lặp

Kết quả: $R0 = 120 = 5 \times 4 \times 3 \times 2 \times 1 = 5!$. Xác nhận đúng. ✓

4.7. Hướng dẫn chạy chương trình

- Bước 1: Mở $emu8086 \rightarrow File \rightarrow Open \rightarrow$ chọn file $vcpusim16.asm$
- Bước 2: Nhấn Emulate (F5) để biên dịch; kiểm tra không có lỗi trong cửa sổ compiler.
- Bước 3: Nhấn Run (F9) để chạy toàn bộ, hoặc Single Step (F8) để chạy từng lệnh x86

- Bước 4: Trên cửa sổ console, nhập phím 1, 2, 3 hoặc 4 để chọn chương trình demo:**
- Bước 5: Quan sát bảng trạng thái VCPU-16 hiển thị sau mỗi bước FDE (Bước | PC | IR | R0–R3 | Z | N | Giai đoạn)**
- Bước 6: Dùng Debugger của emu8086 (cửa sổ Memory Map) để kiểm tra trực tiếp vùng nhớ vcpu_MEM trong DS**

CHƯƠNG 5. ĐÁNH GIÁ VÀ KẾT LUẬN

5.1. Kết quả đạt được

Bảng 5.1: Tổng hợp kết quả bài tập lớn

Hạng mục	Kết quả	Minh chứng
Thiết kế ISA VCPU-16 (16 lệnh, mã hóa 16-bit cố định)	Hoàn thành	Bảng 2.2; file vcpusim16.asm
Xây dựng trình giả lập CPU ảo trên emu8086	Hoàn thành	vcpusim16.asm (~500 dòng)
Hiển thị từng bước FDE kèm tên Giai đoạn	Hoàn thành	Thủ tục display_state
Demo 1: Tính tổng $1+2+3+4+5 = 15$	Đúng (R0=15)	Bảng 4.3
Demo 2: Tìm max $[3,7,2,9,5] = 9$	Đúng (R0=9)	Mục 4.4
Demo 3: Đếm ngược $5 \rightarrow 0$	Đúng (R0=0)	Mục 4.5
Demo 4: Giai thừa $5! = 120$	Đúng (R0=120)	Bảng 4.4
Mã nguồn có comment tiếng Việt đầy đủ	Hoàn thành	Phụ lục A + GitHub

5.2. Ưu điểm và hạn chế

Về ưu điểm, VCPU-16 Simulator đạt được mục tiêu trực quan hóa hoàn toàn: người quan sát thấy PC tăng từng bước, IR nhận mã lệnh 16-bit, bộ giải mã tách opcode bằng SHR/AND, ALU thực thi phép toán và FLAGS cập nhật chính xác sau mỗi lệnh. ISA được thiết kế từ đầu, nhóm kiểm soát hoàn toàn định dạng nhị phân. Phân vùng bộ nhớ Code/Data/Stack minh họa đúng mô hình segmentation. Mã nguồn có comment tiếng Việt chi tiết phục vụ mục đích học thuật.

Về hạn chế, VCPU-16 hiện chưa mô phỏng Cache (do đó không thể minh họa hiệu ứng hit/miss), chưa có Pipeline (mỗi chu kỳ chỉ xử lý một lệnh), tập lệnh chưa có MUL/DIV (phải mô phỏng qua vòng lặp), và bộ nhớ ảo 128 bytes còn nhỏ so với nhu cầu thực tế.

5.3. So sánh VCPU-16 với kiến trúc thực tế

Bảng 5.2: So sánh VCPU-16 với Intel 8086 và ARM Cortex-A53

Tiêu chí	VCPU-16	Intel 8086	ARM Cortex-A53
Loại mô phỏng	Simulator thuần túy	Emulator (emu8086)	Phần cứng thực
Kiểu ISA	Tự thiết kế (giáo dục)	CISC	RISC
Độ rộng bus dữ liệu	16-bit	16-bit	64-bit

Số lệnh ISA	16	~200+	~500+
Bộ nhớ địa chỉ	128 bytes	1 MB (20-bit)	Lên đến hàng TB (64-bit)
Pipeline	Không	Không	8 giai đoạn in-order
Cache	Không	Không	L1/L2/L3 nhiều cấp
Xử lý ngắt	Ngắt phần mềm	Có (INT 0–255)	Có (IRQ, FIQ, SError)
Ngăn xếp	PUSH/POP 16 ô	SS:SP (1 MB)	SP, LR theo ABI

5.4. Phân tích Pipeline – Hướng nâng cấp quan trọng

Pipeline (đường ống) là kỹ thuật cho phép CPU xử lý nhiều lệnh chồng chéo nhau theo từng giai đoạn, tương tự dây chuyền sản xuất công nghiệp. Kiến trúc Pipeline 5 giai đoạn kinh điển (theo Patterson & Hennessy [1]) gồm: IF (Instruction Fetch), ID (Instruction Decode), EX (Execute), MEM (Memory Access), WB (Write Back).

Bảng dưới so sánh việc thực thi 4 lệnh đầu của Demo 1 theo hai mô hình: VCPU-16 hiện tại (tuần tự) và VCPU-16 giả định có Pipeline 5 giai đoạn:

Bảng 5.3: So sánh thực thi tuần tự và Pipeline 5 giai đoạn (4 lệnh Demo 1)

Mô hình	Lệnh	CK 1	CK 2	CK 3	CK 4	CK 5	CK 6	CK 7	CK 8	CK 9	CK 10
Tuần tự (VCPU-16)	LOADI R0,0	IF	ID	EX							
	LOADI R1,1				IF	ID	EX				
Pipeline (nâng cấp)	LOADI R0,0	IF	ID	EX	MEM	WB					
	LOADI R1,1		IF	ID	EX	MEM	WB				

Nhận xét: Với mô hình tuần tự của VCPU-16, 4 lệnh cần $4 \times 3 = 12$ chu kỳ (mỗi lệnh cần IF+ID+EX). Với Pipeline 5 giai đoạn lý tưởng, 4 lệnh chỉ cần $5+3 = 8$ chu kỳ. Với n lệnh nhiều, speedup lý thuyết đạt tối đa 5 lần. Tuy nhiên thực tế thấp hơn do các hazard: data hazard (lệnh sau dùng kết quả lệnh trước chưa xong), control hazard (lệnh nhảy JNZ làm vô hiệu các lệnh đang trong pipeline). Đây là hướng nâng cấp chính của VCPU-16 trong tương lai.

5.5. Khả năng mở rộng và ứng dụng thực tế

- Thêm lệnh MUL, DIV, AND, OR, XOR, NOT, SHL, SHR để hỗ trợ đầy đủ phép toán bit và nhân/chia. Nâng cấp ALU:
- Tăng từ 64 lên 256 hoặc 1024 ô nhớ; thêm cơ chế địa chỉ gián tiếp và địa chỉ tương đối. Mở rộng bộ nhớ:
- Thiết kế lại vòng lặp FDE thành 5 giai đoạn song song IF/ID/EX/MEM/WB, thêm logic phát hiện và xử lý hazard. Cài đặt Pipeline:
- Thêm tầng cache L1 giữa CPU và vcpu_MEM; mô phỏng hit/miss và đo hit rate cho các chương trình demo. Mô phỏng Cache:
- Xây dựng chương trình dịch từ mã nguồn Assembly VCPU-16 (dạng text) sang mã máy hex, thay vì nạp thủ công. Viết Assembler:

5.6. Kết luận

Qua đề tài "Trình giả lập CPU ảo VCPU-16 + tập lệnh hợp ngữ", nhóm đã hoàn thành tất cả các mục tiêu đề ra: thiết kế ISA 16 lệnh, xây dựng trình giả lập hoạt động đầy đủ, và kiểm thử với 4 chương trình demo cho kết quả chính xác.

Điều quan trọng nhất đạt được là sự hiểu biết sâu sắc, không còn mơ hồ: CPU không còn là "hộp đen". Mỗi lần chạy một bước, nhóm thấy rõ PC tăng lên 1, IR nhận mã lệnh 16-bit, bộ giải mã tách OP CODE bằng SHR/AND, ALU thực hiện phép toán, và FLAGS cập nhật chính xác. Đây chính xác là chu trình Fetch-Decode-Execute mà mọi CPU từ Intel 8086 đến Core i9, từ ARM Cortex-A53 đến Apple M4, đều thực hiện hàng tỉ lần mỗi giây.

Nhóm xin chân thành cảm ơn giảng viên hướng dẫn đã định hướng và hỗ trợ trong suốt quá trình thực hiện bài tập lớn.

TÀI LIỆU THAM KHẢO

- [1] Patterson, D. A., & Hennessy, J. L. (2017). Computer Organization and Design: The Hardware/Software Interface (5th ed.). Morgan Kaufmann Publishers. ISBN: 978-0128122754.
- [2] Stallings, W. (2015). Computer Organization and Architecture: Designing for Performance (10th ed.). Pearson Education. ISBN: 978-0134101613.
- [3] Tanenbaum, A. S. (2012). Structured Computer Organization (6th ed.). Prentice Hall. ISBN: 978-0132916523.
- [4] Intel Corporation. (1979). Intel 8086 Family Users Manual. Intel Corporation. Tài liệu kỹ thuật chính thức.
- [5] emu8086 Microprocessor Emulator. (2024). Documentation and User Guide. Truy cập tại: <https://emu8086-microprocessor-emulator.software.informer.com/>
- [6] Nand2Tetris Project. (2024). Build a Modern Computer from First Principles. The Hebrew University of Jerusalem. Truy cập tại: <https://www.nand2tetris.org/>
- [7] Ben Eater. (2023). Building an 8-bit breadboard computer! Series. Truy cập tại: <https://eater.net/8bit>
- [8] Hyde, R. (2010). The Art of Assembly Language (2nd ed.). No Starch Press. ISBN: 978-1593272074.

PHỤ LỤC A – MÃ NGUỒN CHỌN LỌC VÀ ĐƯỜNG DẪN ĐẦY ĐỦ

Mã nguồn đầy đủ (~500 dòng) được lưu trữ tại GitHub và có thể truy cập qua QR code hoặc đường dẫn ở cuối phụ lục này. Dưới đây là các đoạn mã quan trọng nhất, có comment tiếng Việt giải thích từng dòng lệnh.

A.1. Khai báo phần cứng ảo (Data Segment)

Đây là phần khai báo toàn bộ "phần cứng" của VCPU-16 dưới dạng biến trong DS của chương trình x86:

```
; == THANH GHI CỦA VCPU-16
=====
vcpu_R0      DW 0          ; Thanh ghi đa năng R0 (16-bit)
vcpu_R1      DW 0          ; Thanh ghi đa năng R1 (16-bit)
vcpu_R2      DW 0          ; Thanh ghi đa năng R2 (16-bit)
vcpu_R3      DW 0          ; Thanh ghi đa năng R3 (16-bit)
vcpu_PC      DW 0          ; Program Counter - địa chỉ lệnh tiếp
theo
vcpu_IR      DW 0          ; Instruction Register - lệnh đang xử
lý
vcpu_FLAGS   DW 0          ; Cờ trạng thái: bit0=Z, bit1=N,
bit2=C
vcpu_SP      DW 15         ; Stack Pointer - đỉnh ngăn xếp (ban
đầu=15)

; == BỘ NHỚ ẢO 128 BYTES
=====
; vcpu_MEM[0..31] = Code Segment (chứa lệnh chương trình)
; vcpu_MEM[32..47] = Data Segment (biến, mảng dữ liệu)
; vcpu_MEM[48..63] = Stack Segment (LIFO - PUSH/POP)
vcpu_MEM     DW 64 DUP(0)   ; 64 word × 2 bytes = 128 bytes ảo

; == BẢNG TRA ĐỊA CHỈ THANH GHI (truy cập động theo số hiệu)
=====
reg_table    DW vcpu_R0, vcpu_R1, vcpu_R2, vcpu_R3
; Dừng: SHL RD,1 → LEA SI,reg_table → MOV SI,[SI+RD] →
[SI]=giá_trị
```

A.2. Vòng lặp FDE chính (run_vcpu)

```
; == THỦ TỤC run_vcpu - Vòng lặp Fetch-Decode-Execute
=====
run_vcpu PROC
vcpu_loop:
    CMP vcpu_running, 1    ; Kiểm tra CPU có đang chạy không
    JNE vcpu_stopped       ; Nếu không (HALT đã thực thi):
thoát
```

```

; — FETCH
MOV  BX, vcpu_PC      ; BX = PC hiện tại
SHL  BX, 1            ; BX = PC × 2 (offset byte trong
mảng word)
LEA  SI, vcpu_MEM
MOV  AX, [SI + BX]    ; AX = lệnh 16-bit tại MEM[PC]
MOV  vcpu_IR, AX      ; Nạp vào Instruction Register
INC  vcpu_PC          ; PC ← PC + 1

; — DECODE
MOV  CX, AX | SHR CX, 12 | AND CX, 0Fh ; OPCODE
MOV  DX, AX | SHR DX, 10 | AND DX, 03h ; RD
MOV  BX, AX | SHR BX, 8  | AND BX, 03h ; RS
AND  AX, 00FFh        ; IMM8

; — EXECUTE - Bảng nhảy theo OPCODE
CMP  CX, 0 | JE exec_halt ; HALT
CMP  CX, 1 | JE exec_loadi ; LOADI
CMP  CX, 5 | JE exec_add ; ADD
CMP  CX, 0Ch | JE exec_jnz ; JNZ
; ... (12 lệnh còn lại tương tự)

CALL display_state ; In trạng thái sau mỗi bước FDE
JMP  vcpu_loop     ; Lặp lại
vcpu_stopped:
; In kết quả cuối và chờ phím
run_vcpu ENDP

```

A.3. Thủ tục exec_add và update_flags_result

```

; == exec_add: Xử lý lệnh ADD Rd, Rs
exec_add:
POP  DX      ; DX = RD (số hiệu thanh ghi đích)
POP  BX      ; BX = RS (số hiệu thanh ghi nguồn)
POP  AX      ; AX = IMM8 (không dùng trong ADD)

; Đọc giá trị Rs từ bộ nhớ x86 qua reg_table
SHL  BX, 1 | LEA SI, reg_table | MOV SI, [SI+BX]
MOV  CX, [SI] ; CX = giá trị của Rs

; Đọc Rd và thực hiện phép cộng
SHL  DX, 1 | LEA SI, reg_table | MOV SI, [SI+DX]
MOV  AX, [SI] ; AX = giá trị hiện tại của Rd
ADD  AX, CX   ; AX = Rd + Rs (phép cộng chính)
MOV  [SI], AX ; Ghi kết quả ngược về thanh ghi Rd

CALL update_flags_result ; Cập nhật FLAGS dựa trên AX
CALL display_state      ; In trạng thái sau lệnh này
JMP  vcpu_loop          ; Tiếp tục vòng lặp FDE

```

```
; === update_flags_result: Cập nhật Z và N dựa trên AX
=====
update_flags_result PROC
    MOV    CX, 0                ; Bắt đầu: xóa toàn bộ FLAGS
    CMP    AX, 0 | JNE @neg | OR CX, 01h ; Z=1 nếu AX=0
@neg:
    TEST   AX, 8000h | JZ @done | OR CX, 02h ; N=1 nếu
bit15=1
@done:
    MOV    vcpu_FLAGS, CX ; Lưu FLAGS vào thanh ghi ảo
    RET
update_flags_result ENDP
```

A.4. Đường dẫn mã nguồn đầy đủ

Thông tin	Chi tiết
GitHub Repository	github.com/[ten-nhom]/vcpu16-simulator
File chính	vcpusim16.asm – Assembly x86 16-bit, chạy trên emu8086
Hướng dẫn tải	git clone https://github.com/hangthunguyen965-cloud/RTL_Kien_Truc_May_Tinh

PHỤ LỤC B – BẢNG MÃ MÁY CÁC CHƯƠNG TRÌNH DEMO

B.1. Demo 1 – Tính tổng (MEM[0..7])

PC	Hex	Nhị phân	Lệnh VCPU-16	Ghi chú
0	0x1000	0001 0000 0000 0000	LOADI R0, 0	R0 = tổng, khởi đầu = 0
1	0x1401	0001 0100 0000 0001	LOADI R1, 1	R1 = biến đếm, bắt đầu từ 1
2	0x1805	0001 1000 0000 0101	LOADI R2, 5	R2 = giới hạn vòng lặp
3	0x5004	0101 0000 0000 0100	ADD R0, R1	Cộng dồn R1 vào tổng R0
4	0x7400	0111 0100 0000 0000	INC R1	Tăng biến đếm lên 1
5	0x9408	1001 0100 0000 1000	CMP R1, R2	So sánh biến đếm với giới hạn
6	0xC003	1100 0000 0000 0011	JNZ 3	Z=0: quay về PC=3 (lặp tiếp)
7	0x0000	0000 0000 0000 0000	HALT	Dừng; kết quả trong R0 = 15

B.2. Demo 4 – Giai thừa 5! = 120 (MEM[0..10])

PC	Hex	Nhị phân	Lệnh VCPU-16	Ghi chú
0	0x1001	0001 0000 0000 0001	LOADI R0, 1	R0 = tích lũy kết quả (= 1)
1	0x1405	0001 0100 0000 0101	LOADI R1, 5	R1 = biến đếm (5→4→3→2→1)
2	0x4200	0100 0010 0000 0000	MOV R2, R0	R2 = R0 cũ (lưu trước khi nhân)
3	0x4C04	0100 1100 0000 0100	MOV R3, R1	R3 = R1 (bộ đếm vòng nhân)
4	0x1000	0001 0000 0000 0000	LOADI R0, 0	R0 = 0 (bộ tích lũy nhân)
5	0x5002	0101 0000 0000 0010	ADD R0, R2	R0 += R2 (cộng lặp R3 lần)
6	0x8C00	1000 1100 0000 0000	DEC R3	R3-- (bộ đếm nhân)
7	0xC005	1100 0000 0000 0101	JNZ 5	R3≠0: lặp phép cộng
8	0x8400	1000 0100 0000 0000	DEC R1	R1-- (giảm biến đếm giai thừa)

9	0xC002	1100 0000 0000 0010	JNZ 2	R1≠0: lặp vòng nhân tiếp
10	0x0000	0000 0000 0000 0000	HALT	Dừng; R0 = 120 = 5!

B.3. Hướng dẫn chạy

-Bước 1: Tải file về và lưu file vào đường dẫn không có dấu cách, ví dụ

C:\emu8086\vcpu

-Bước 2: Mở emu8086 vào **File** → **Open** → chọn file **vcpusim16.asm**

-Bước 3: Nhấn **F5** (Assemble + Load) – kiểm tra không có lỗi

-Bước 4: Nhấn **F9** (Run) – cửa sổ console hiện menu chọn demo

-Bước 5: Nhấn phím **1 / 2 / 3 / 4** để chạy demo tương ứng

-Bước 6: Đọc kết quả: Demo 1 → R0=15, Demo 2 → R0=9, Demo 3 → R0=0, Demo 4 → R0=120

-*Bước 7(Tùy chọn)*: Nhấn **F8** thay vì F9 để chạy từng bước, xem thanh ghi thay đổi